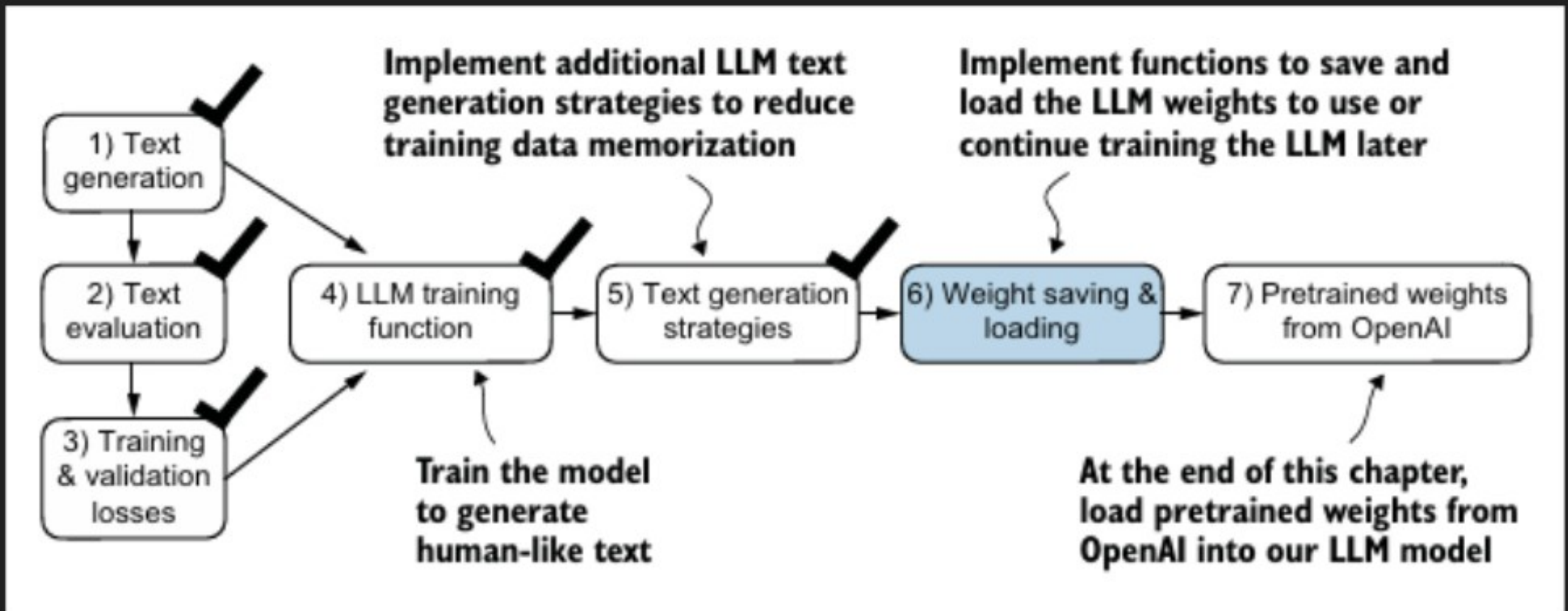


lecture:31 - loading pre-trained weight from OpenAI GPT-2



↳ using OpenAI openly shared weights

```
gpt_download3.py

def download_and_load_gpt2(model_size, models_dir):
    # Validate model size
    allowed_sizes = ("124M", "355M", "774M", "1558M")
    if model_size not in allowed_sizes:
        raise ValueError(f"Model size not in {allowed_sizes}")

    # Define paths
    model_dir = os.path.join(models_dir, model_size)
    base_url = "https://openaipublic.blob.core.windows.net/gpt-2/models"
    filenames = [
        "checkpoint",
        "encoder.json",
        "hparams.json",
        "model.ckpt.data-00000-of-00001",
        "model.ckpt.index",
        "model.ckpt.meta",
        "vocab.bpe",
    ]

    # Download files
    os.makedirs(model_dir, exist_ok=True)
    for filename in filenames:
        file_url = os.path.join(base_url, model_size, filename)
        file_path = os.path.join(model_dir, filename)
        download_file(file_url, file_path)

    ## We have reached here until now —> we have downloaded the files on our local machine.

    # Load settings and params
    tf_ckpt_path = tf.train.latest_checkpoint(model_dir)
    settings = json.load(open(os.path.join(model_dir, "hparams.json")))
    params = load_gpt2_params_from_tf_ckpt(tf_ckpt_path, settings)

    return settings, params
```



```
def load_gpt2_params_from_tf_ckpt(ckpt_path, settings):
    # Initialize parameters dictionary with empty blocks for each layer
    params: dict[str, Any] = {"blocks": [{} for _ in range(settings["n_layer"])]}

    # Iterate over each variable in the checkpoint
    for name, _ in tf.train.list_variables(ckpt_path):
        # Load the variable and remove singleton dimensions
        variable_array = np.squeeze(tf.train.load_variable(ckpt_path, name))

        # Process the variable name to extract relevant parts
        variable_name_parts = name.split("/")[1:] # Skip the 'model/' prefix

        # Identify the target dictionary for the variable
        target_dict: dict[str, Any] = params
        if variable_name_parts[0].startswith("h"):
            layer_number = int(variable_name_parts[0][1:])
            target_dict = params["blocks"][layer_number]

        # Recursively access or create nested dictionaries
        for key in variable_name_parts[1:-1]:
            if key not in target_dict:
                target_dict[key] = {}
            next_value = target_dict[key]
            if isinstance(next_value, dict):
                target_dict = next_value

        # Assign the variable array to the last key
        last_key = variable_name_parts[-1]
        target_dict[last_key] = variable_array

    return params
```

Parameter Dictionary key's

① wte: token embedding

② wpe: positional embedding

③ blocks

④ g: Final normal. scale

⑤ b: Final normal. shift

768
50257
768
1024

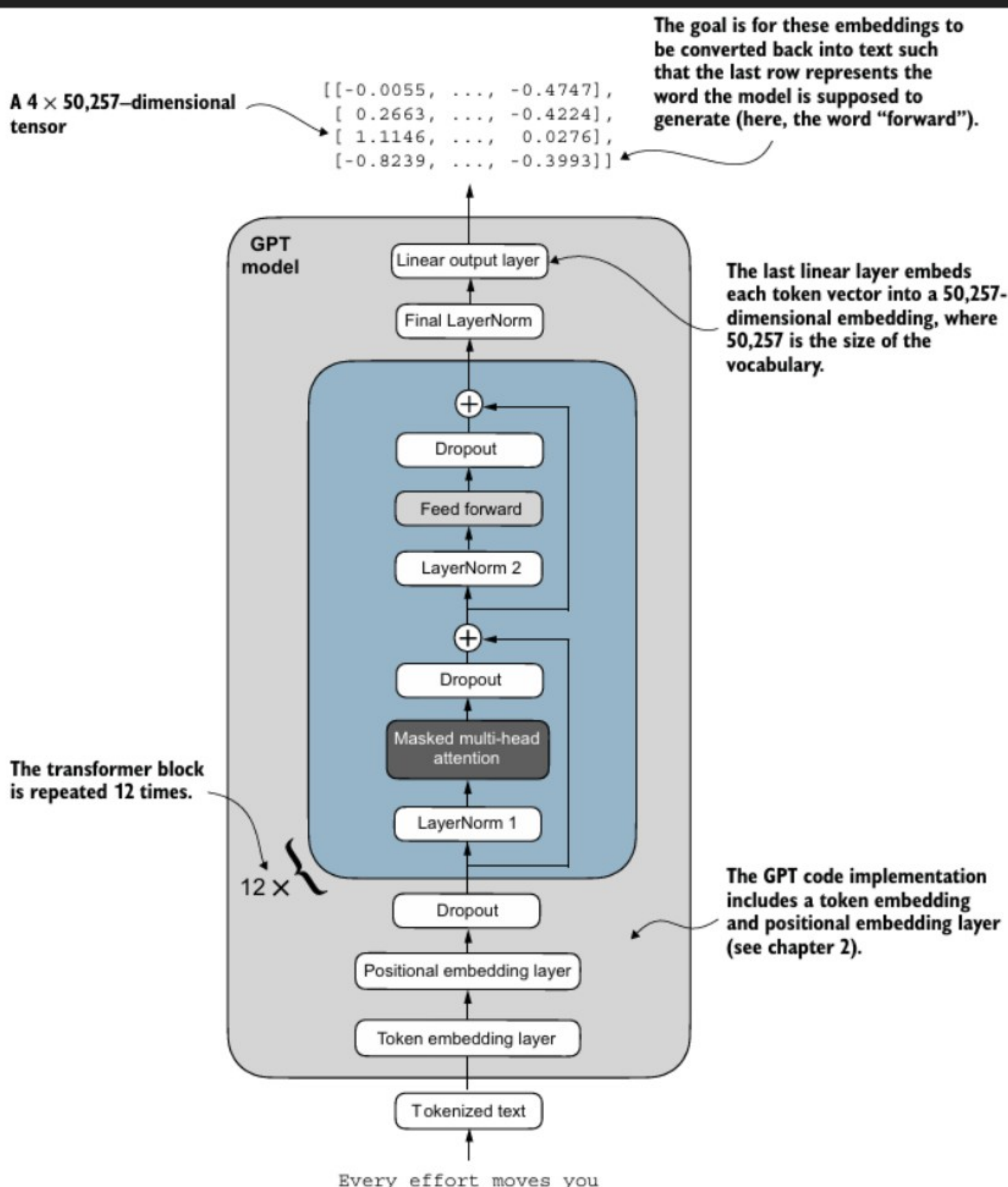


Figure 4.15 An overview of the GPT model architecture showing the flow of data through the GPT model. Starting from the bottom, tokenized text is first converted into token embeddings, which are then augmented with positional embeddings. This combined information forms a tensor that is passed through a series of transformer blocks shown in the center (each containing multi-head attention and feed forward neural network layers with dropout and layer normalization), which are stacked on top of each other and repeated 12 times.

* Attention layer in transformer blocks:

$\text{transformer} / h_0 / \text{attn} / \text{c-attn} / w$
 $\text{transformer} / h_0 / \text{attn} / \text{c-attn} / b$

similarly for $h_1, h_2, \dots$
 Q, K, V

* Feedforward neural network weight in transformer blocks:

$\text{transformer} / h_0 / \text{mlp} / \text{c-tc} / w$
 $\text{transformer} / h_0 / \text{mlp} / \text{c-tc} / b$

$\text{mlp} \rightarrow \text{multilayer perceptron}$

similarly for $h_1, h_2, \dots$

$\text{transformer} / h_0 / \text{mlp} / \text{c-proj} / w$
 $\text{transformer} / h_0 / \text{mlp} / \text{c-proj} / b$

$\text{mlp} \rightarrow \text{multilayer perceptron}$

similarly for $h_1, h_2, \dots$

* Output projection layer in transfer block:

$\text{transformer} / h_0 / \text{attn} / \text{c-proj} / w$

similarly for $h_1, h_2, \dots$

* Layer normalization:

$\text{layer } I \left\{ \begin{array}{l} \text{transformer} / h_0 / l_{n-1} / g \rightarrow \text{layer norm. scale} \\ \text{transformer} / h_0 / l_{n-1} / b \rightarrow \text{layer norm. shift} \end{array} \right.$

$\text{layer } I \left\{ \begin{array}{l} \text{transformer} / h_0 / l_{n-2} / g \rightarrow \text{layer norm. scale} \\ \text{transformer} / h_0 / l_{n-2} / b \rightarrow \text{layer norm. shift} \end{array} \right.$

similarly for $h_1, h_2, \dots$